

Clean Code Deep Dive – JUnit ComparisonCompactor

“Clean code always looks like it was written by someone who cares.” – Robert C. Martin

1. Overview

This document explains how the JUnit ComparisonCompactor evolves from good code into excellent code through refactoring. We focus on readability, naming, and removing ambiguity.

2. Bad Code Example (Original Style – Hard to Read)

```
public String compact(String message) {
    if (fExpected == null || fActual == null || areStringsEqual())
        return Assert.format(message, fExpected, fActual);

    findCommonPrefix();
    findCommonSuffix();

    String expected = compactString(fExpected);
    String actual = compactString(fActual);

    return Assert.format(message, expected, actual);
}

private String compactString(String source) {
    String result = "[" +
        source.substring(fPrefix, source.length() - fSuffix + 1) +
        "]";

    if (fPrefix > 0)
        result = computeCommonPrefix() + result;

    if (fSuffix > 0)
        result = result + computeCommonSuffix();

    return result;
}
```

Problems:

- Unclear variable naming (fExpected, fSuffix)
- Magic numbers (+1)
- Mixed responsibilities
- Hard to understand intent

3. Good Code Example (Refactored – Clean & Clear)

```
public String formatCompactedComparison(String message) {
    if (!canBeCompacted()) {
        return Assert.format(message, expected, actual);
    }

    findCommonPrefixAndSuffix();

    String compactExpected = compact(expected);
    String compactActual = compact(actual);

    return Assert.format(message, compactExpected, compactActual);
}
```

```
private boolean canBeCompacted() {
    return expected != null && actual != null && !expected.equals(actual);
}
```

Improvements:

- Intent is clear
- Naming is meaningful
- Logic is separated
- Easier to maintain

4. Removing Magic Numbers (+1 Problem)

```
private char charFromEnd(String text, int offsetFromEnd) {
    return text.charAt(text.length() - offsetFromEnd - 1);
}
```

Instead of using confusing +1 everywhere, we centralize the logic in one clear function.

5. Naming Improvement (suffixIndex → suffixLength)

suffixIndex was misleading because it stored a length, not an index. Renaming improves clarity and prevents logical mistakes.

6. Final Clean Structure (Full Example)

```
private String compact(String s) {
    return new StringBuilder()
        .append(startingEllipsis())
        .append(startingContext())
        .append("[")
        .append(delta(s))
        .append("]")
        .append(endingContext())
        .append(endingEllipsis())
        .toString();
}

private String delta(String s) {
    int start = prefixLength;
    int end = s.length() - suffixLength;
    return s.substring(start, end);
}
```

Now the code reads like a story: build prefix → show difference → build suffix.

7. Step-by-Step Refactoring Mindset

- Start with working but messy code
- Improve naming (remove f prefix)
- Extract meaningful functions
- Remove magic numbers

- Separate logic (analysis vs synthesis)
- Continuously simplify

8. Quotes

"Clean code reads like well-written prose." – Robert C. Martin

"Simplicity is the ultimate sophistication." – Leonardo da Vinci

"Programs must be written for people to read." – Harold Abelson

9. Conclusion

Clean code is not about writing perfect code from the start, but refining it step by step until it becomes simple, readable, and maintainable.